ASSIGNMENT 4 - CO4

TASK 3 - MODERN FILE SYSTEMS

Choosing, Inspecting and Evaluating File Systems | 9 Marks | K5 - Evaluate

1. Purpose and Scope

Task 3 asks for a comparison of FAT32, NTFS, ext4, XFS, Btrfs, ZFS and APFS; an explanation of crash consistency and ext4 journaling modes; file-system choices for four devices; and inspection evidence from a real Linux file system. The alternative concept compares ext4 journaling with a copy-on-write file system such as Btrfs or ZFS.

The size figures below are format-level or theoretical limits where stated. Actual usable limits depend on operating-system version, block/cluster size, driver, architecture, volume layout and vendor support. Binary units are used: TiB = 2^40 bytes, EiB = 2^60 bytes, ZiB = 2^70 bytes.

2. Task 3.1 - Maximum File and Volume Sizes

File system	Maximum individual file size	Maximum volume / pool size	Important qualification
FAT32	4 GiB - 1 byte (4,294,967,295 bytes).	Common 512-byte-sector implementations are limited to about 2 TiB; format limits depend on sector/cluster geometry.	Microsoft's format comparison lists a 4 GiB file limit and 2^32 allocation blocks.
NTFS	64-bit file-size field; Microsoft lists 2^64 - 1 bytes as the file-size limit.	With the default 4 KiB clusters, Microsoft's current table lists 16 TB. Larger cluster sizes support larger volumes, up to 8 PB in the documented table.	The practical limit depends on cluster size and Windows version.
ext4	Up to 16 TiB for a 4 KiB block size with extents.	64-bit ext4 block counts have very large theoretical limits; current kernel documentation lists up to 64 ZiB at 4 KiB blocks. Distribution-certified limits can be much lower.	File size, block size, extents and the 64-bit feature affect limits.
XFS	Theoretical maximum 8 EiB.	Theoretical maximum 8 EiB.	Supported production limits are lower on some distributions; for example, RHEL's supported volume limit is 1 PB in current releases.
Btrfs	Inherent limit 16 EiB; practical Linux VFS file-length limit 8 EiB.	Theoretical filesystem limit is very large (commonly documented at 16 EiB); practical limit depends on kernel, devices and metadata.	Btrfs documentation explicitly distinguishes its 2^64-byte inherent file length from Linux VFS's 2^63-byte limit.
ZFS	Theoretical maximum 16 EiB.	Theoretical filesystem size 16 EiB; a zpool's theoretical maximum is commonly listed as 256 ZiB.	These are design limits, not normal single-device or tested deployment sizes.
APFS	Apple's archived guide lists 2^63 bytes (8 EiB) as maximum file size.	Apple documents 2^63 allocation blocks; the byte ceiling depends on allocation-block size and implementation. A single general byte maximum is not stated in the current user guide.	APFS volume/container sizing and available capacity are constrained by the device and macOS/iOS implementation.

The limits are not directly interchangeable: FAT32 and NTFS limits vary with volume geometry and OS; Linux file systems have kernel and distribution support limits; ZFS uses pools rather than a single traditional partition; APFS volumes share free space inside a container. Use the values as a comparison, not as a promise that every system can create a volume at the theoretical maximum.

3. Task 3.1 - Journaling, Snapshots, Checksums and Copy-on-Write

File system	Journaling / crash consistency	Snapshots	Checksums	Copy-on-write
FAT32	No journal; metadata updates can be left inconsistent after an unexpected shutdown.	No native snapshots.	No general file-data or metadata checksum facility.	No.
NTFS	Metadata transaction log ($LogFile); supports metadata recovery, not full user-file data journaling.	Volume snapshots are generally provided by Windows Volume Shadow Copy Service (VSS), not as native NTFS snapshots.	No general end-to-end file-data checksum by default.	No filesystem-wide COW; VSS uses a separate snapshot mechanism.
ext4	JBD2 journal; normally metadata journaling. Three data modes are explained in Section 4.	No native filesystem snapshots; use LVM/device-level snapshots or backup software.	Metadata checksums are supported; ordinary file data is not end-to-end checksummed by default.	No general COW data model.
XFS	Metadata write-ahead log; transaction recovery on mount.	No native snapshot mechanism; use LVM or storage-layer snapshots.	CRC32c checksums validate metadata structures, not a full end-to-end checksum of every user-data block.	Optional reflink support shares extents and copies changed blocks on write; not a full COW filesystem.
Btrfs	COW transactions and tree-log replay; not a traditional journal.	Native read-only and writable subvolume snapshots.	Checksums for data and metadata; can detect silent corruption and repair when redundant good copies exist.	Yes; data and metadata are updated by writing new blocks and committing new tree roots.
ZFS	COW transaction groups; ZFS Intent Log (ZIL) records synchronous writes. Not a conventional metadata journal.	Native instantaneous snapshots; space grows as referenced blocks later diverge.	End-to-end checksums for data and metadata; redundancy can allow self-healing on read/scrub.	Yes; blocks in use are not overwritten in place.
APFS	Copy-on-write metadata and atomic updates provide crash protection; it does not use a traditional journal.	Native snapshots, clones and space sharing.	Fletcher checksum for filesystem metadata; ordinary user-data checksumming is not a general default APFS feature.	Yes; metadata is COW, and file/directory clones share blocks until modified.

Interpretation: journaling records changes so the file system can recover a consistent metadata structure after a crash. COW writes new blocks and then commits references to them; the previous committed tree remains available until the new root is committed. Snapshots and COW are related but not identical: COW enables snapshots to retain old blocks efficiently, while snapshot tools decide which historical roots remain pinned.

4. Task 3.2 - Power Failure, Journaling and Consistency

If power fails during a write, dirty data or metadata may still be in memory or in a device cache. Without a recovery mechanism, directory entries, allocation maps, inode metadata and data blocks can disagree. The next mount may discover orphaned blocks, missing directory entries or metadata pointing to incomplete data. A filesystem's consistency mechanism reduces structural corruption, but it cannot automatically guarantee that an application-level transaction or every last user-data byte was committed.

4.1 ext4 journaling modes

Mode	What is journaled and write order	Recovery / performance trade-off
data=journal	Both file data and metadata are written through the journal before reaching their final locations.	Strongest data journaling of the three modes, but usually slowest because data is written twice. It disables delayed allocation and O_DIRECT. Current kernel documentation marks this mode deprecated, with removal planned for January 2028.
data=ordered (default on typical Linux ext4 mounts)	Metadata is journaled. Associated data blocks are written to their home locations before the metadata transaction is committed.	Good balance: metadata recovery is available and data is ordered before metadata, but it does not journal all file data. Application durability still depends on fsync/fdatasync and correct write ordering.
data=writeback	Metadata is journaled, but there is no ordering guarantee that file data reaches its home location before metadata is committed.	Often fastest, but after a crash, recently modified files can contain stale or inconsistent data even though metadata structures can be recovered.

After an unclean shutdown, ext4 replays committed journal transactions and discards incomplete ones. This restores filesystem metadata to a structurally consistent state. In ordered mode, associated data is flushed before metadata commit; in writeback mode, data may reach disk later; in journal mode, data and metadata both pass through the journal. None of these modes replaces application-level transactions or backups.

4.2 COW consistency

A COW filesystem does not overwrite a block that is still part of the committed filesystem tree. It allocates new space for changed content, writes new metadata nodes containing the updated references, and finally commits a new root/transaction. If power fails before the new root is committed, the older committed root remains usable. If it fails after commit, the new tree is selected. Btrfs replays its tree log where needed; ZFS imports the pool from its latest valid transaction group and uses the ZIL to replay synchronous writes that require it.

COW protects filesystem structure from partially overwritten metadata, but does not make every asynchronous application write durable. Applications that require durable transactions must still use fsync/fdatasync or the database's documented flush protocol. A snapshot is not a backup against pool/device loss unless it is replicated to independent storage.

5. Task 3.3 - Choose a File System for Each Device

Device	Recommended choice	Justification and limitation
Pen drive used on Windows and Linux	FAT32 for maximum basic interchange compatibility.	Widely supported by Windows and Linux without special drivers. The 4 GiB - 1 byte file limit is a serious limitation. If files larger than 4 GiB are required and the Linux systems have NTFS read/write support, NTFS is an alternative; exFAT is also commonly used but is outside the seven filesystems this task asks us to compare.
Linux database server	ext4 using the default data=ordered mode, with database WAL and fsync discipline.	Mature Linux support, journaled metadata, predictable operations and broad tooling. The database must flush its own log/data correctly; a filesystem journal alone does not make transactions durable. XFS is also a valid choice for large and concurrent workloads, but ext4 is selected for a general-purpose Linux database server.
Laptop	APFS for a macOS laptop; ext4 for a Linux laptop.	APFS is designed for Apple platforms and flash storage, with encryption, COW metadata, snapshots, clones and space sharing. If the laptop runs Linux, ext4 is the natural supported choice; if it runs Windows, NTFS is the normal system filesystem. The question does not specify laptop OS, so the recommendation is conditional on its operating system.
Backup server that needs snapshots	ZFS, configured with redundant storage and scheduled snapshots/scrubs.	Native snapshots, end-to-end checksums, compression, replication and scrub support make ZFS suitable for backup datasets. Redundancy is required for self-healing; checksums alone detect corruption but cannot repair it without a good replica. Btrfs is a Linux-native alternative with COW snapshots and data/metadata checksums.

Decision: use the filesystem that is native and well-supported on the target operating system, not just the one with the largest theoretical limit. Cross-platform removable storage prioritises compatibility; a database server prioritises stable low-latency writes and application durability; a laptop prioritises platform integration and encryption; a backup server prioritises snapshots, integrity verification and recovery.

6. Task 3.4 - Inspect a Real Linux File System

The assignment explicitly asks for `stat`, `df -i`, `ls -i` and `tune2fs -l` or `dumpe2fs -h` to show inodes, block size and free space. Run the read-only inspection script in Ubuntu/WSL against a directory on the Linux filesystem you want to inspect. The script records the mount source and type, inode numbers, filesystem block statistics, free inodes and ext filesystem details where available.

6.1 Manual commands

```
# 1. Identify the mount and filesystem type
findmnt -T .

# 2. File/directory metadata and inode
stat .

# 3. Filesystem block size, free space and inode information
stat -f .
df -hT .
df -i .

# 4. Inode numbers for files in the current directory
ls -lai .

# 5. If the inspected mount is ext2/ext3/ext4:
sudo tune2fs -l /dev/<ext-filesystem-device>
# Alternative:
sudo dumpe2fs -h /dev/<ext-filesystem-device>
```

The block-device path must be the actual ext filesystem device identified by `findmnt`; do not guess `/dev/sda` or run these commands on a partition that contains important data without confirming it. `tune2fs

-l` and `dumpe2fs -h` are inspection operations; do not use formatting or repair options.

6.2 Automated evidence

Save `inspect_filesystem.sh` beside this report. In Ubuntu/WSL, run `chmod +x inspect_filesystem.sh` and `./inspect_filesystem.sh /path/to/target`. The default output folder is `task3_fs_evidence/`, containing `filesystem_inspection.txt` and `filesystem_specific_details.txt`. If the target is ext4, the script attempts `sudo tune2fs -l`; for Btrfs, XFS and ZFS it uses their read-only information tools when installed.

A WSL path under `/mnt/c` may be backed by a Windows/DrvFS or virtual filesystem, not a native ext4 block device. In that case, `stat`, `df` and `ls -i` still show the mounted view, but `tune2fs` cannot inspect a Windows volume. To demonstrate ext filesystem superblock/inode information, inspect a real ext4 partition or a disposable ext4 virtual disk inside Linux.

7. Alternative Concept - Use Btrfs or ZFS Instead of ext4

The alternative asks how the database-server design changes if ext4 journaling is replaced by a COW filesystem such as Btrfs or ZFS. The main differences are write amplification, snapshot space retention, data checksums and crash recovery.

7.1 Small random-update write time

Aspect	ext4 with data=ordered	Btrfs / ZFS (COW)
Data write path	Typically writes changed data blocks to their home location and journals the related metadata transaction.	Writes changed data to new blocks, then writes new/updated metadata nodes and commits a new root/transaction.
Small random writes	Often lower write amplification for in-place 4 KiB updates; metadata still passes through the journal.	Can be slower when a tiny update causes data-block allocation plus COW metadata-tree updates. Large recordsize/extent settings can make small writes more expensive.
Durability setting	Journal mode and application fsync/WAL determine ordering and persistence.	COW provides atomic metadata-root updates; synchronous application writes are protected through Btrfs log/transaction handling or ZFS ZIL.
Tuning	Use database WAL, fsync, barriers, appropriate I/O scheduler and tested mount options.	Match Btrfs/ZFS data record/extent size to database I/O; avoid oversized records for small random updates; monitor pool free space and snapshot retention.

A theoretical comparison of one modified block: ext4's ordered mode writes the data block and journals metadata; COW systems allocate a new data block and rewrite the path of metadata nodes to the root. For a balanced metadata tree with N entries and height h = O(log N), a COW update can require O(log N) metadata-node writes in addition to the data write. Real write latency depends on cache hits, batching, record size, compression, SSD behaviour, sync policy and database `fsync` frequency. There is no universal guarantee that COW is slower for every workload.

7.2 Snapshot space and checksums

COW snapshots initially reference existing blocks and therefore require little extra data space at creation. As live data changes or is deleted, old blocks that are still referenced by snapshots cannot be freed. Snapshot space therefore grows with the unique blocks retained by all snapshots, not simply with the logical dataset size.

Illustrative calculation: a 1 TiB dataset has a read-only snapshot. If 6% of the original allocated blocks are overwritten or deleted while the snapshot is retained, about 0.06 x 1 TiB = 61.44 GiB of old block content may remain pinned by that snapshot, plus metadata and any new data blocks. If several snapshots overlap, the physical cost is the union of unique old blocks they retain, not the sum of every snapshot's logical size.

Btrfs checksums data and metadata (CRC32C by default for data); ZFS stores end-to-end checksums for data and metadata. Checksums add CPU work and metadata storage but detect silent corruption. With redundant copies or RAID protection, the filesystem can sometimes repair a damaged block using a verified good copy. ext4 also supports metadata checksums, but it does not provide the same general end-to-end data-checksum model for ordinary user data.

7.3 Recovery after power failure

Filesystem	Recovery mechanism	What to expect
ext4	Replay committed JBD2 journal transactions; discard incomplete ones.	Usually fast metadata recovery proportional to journal work, not a scan of the entire volume. Ordered/writeback modes do not guarantee all recently written user data was committed.
Btrfs	COW transactions and tree-log replay where needed.	Mount recovers from the last consistent committed tree and replays appropriate log items. It avoids in-place partial tree updates, but device damage or uncorrectable checksum errors still need repair/backup.
ZFS	Import the pool from the latest valid transaction group; replay ZIL records for synchronous writes requiring replay.	The committed tree remains consistent. Recovery depends on pending transaction/log work and pool/device state; checksums and redundancy aid detection/repair.

Do not test power-loss recovery by pulling power from a physical database server. If a measured comparison is required, use a disposable virtual machine with a throwaway test volume and controlled forced-reset procedure, then record mount/recovery time and verify file checksums after reboot. A normal graceful reboot does not simulate power failure.

8. Complexity and Trade-off Summary

Operation	ext4 journaling	Btrfs / ZFS COW
Append/update a data block	Data write + journaled metadata transaction; journal work proportional to changed metadata in transaction.	New data block + changed metadata-tree path; roughly O(log N) metadata-node updates for a balanced tree.
Create snapshot	Not natively supported by ext4; storage/LVM layer snapshot complexity and cost depend on implementation.	Near-constant-time root/subvolume reference operation initially; O(1) new data space at creation, later space O(C) for unique changed blocks retained.
Crash recovery	Journal replay work O(J), where J is the committed journal records/blocks to replay.	Log/transaction recovery work proportional to pending log/transaction records, not a full scan of all user data.
Integrity check	Metadata checksums if enabled; no standard end-to-end data checksum.	Data and metadata checksum cost is O(B) for B bytes verified; repair requires redundancy.

These Big-O descriptions model metadata work at the filesystem level; they do not predict latency in seconds. Actual performance must be measured on identical hardware, mount settings, block size, dataset size, cache state and sync policy.

9. Implementation and Evidence Procedure

Task 3 does not include a C `Complete the Code` block. The required implementation evidence is Linux command output from inspecting a real filesystem. Two shell utilities are included: a read-only inspector and an optional `fio` comparison script for small random writes.

9.1 Inspect the Linux filesystem

```
chmod +x inspect_filesystem.sh
./inspect_filesystem.sh .

# Or inspect a mounted Linux volume:
./inspect_filesystem.sh /mnt/ext4
```

Review `task3_fs_evidence/filesystem_inspection.txt` and `filesystem_specific_details.txt`. Preserve the actual outputs and take a terminal screenshot. On ext4, verify block size, inode count, free blocks and free inodes in the `tune2fs -l` output.

9.2 Optional small random-write comparison

```
sudo apt update
sudo apt install fio

# Use existing writable mount directories on ext4 and Btrfs/ZFS:
EXT4_DIR=/mnt/ext4 COW_DIR=/mnt/btrfs ./benchmark_small_writes.sh
```

The script uses 4 KiB random writes with direct I/O and fdatasync after each write, writes into temporary directories, saves JSON results and removes the test files. Repeat each filesystem test at least three times. Do not compare buffered I/O on one filesystem with direct I/O on another; the script does not silently switch to a buffered fallback.

9.3 Snapshot and power-failure evidence

For snapshot-space measurement, use a disposable Btrfs subvolume or ZFS dataset, record `btrfs filesystem du` or `zfs list` before and after a controlled amount of data changes, and record the amount held by snapshots. For recovery time, use only a disposable VM and a test disk. Save the recovery log and verify sample file hashes after reboot. The report's space and recovery discussion is analytical; no snapshot or crash test has been fabricated.

10. Final Recommendation

For a general Linux database server, ext4 with data=ordered and correct database WAL/fsync settings is a sound default when simplicity, low overhead and mature support are priorities. For a backup server that requires native snapshots and end-to-end integrity checking, choose ZFS with redundant storage, scheduled snapshots, periodic scrubs and tested replication. Btrfs is a viable Linux-native COW alternative. FAT32 is suitable for a small cross-platform pen drive only when no individual file exceeds 4 GiB; for a macOS laptop use APFS, while Linux/Windows laptop choices should follow the installed operating system.

The choice between ext4 and COW storage for a database must be based on the database's actual small-random-write latency, sync behaviour, snapshot retention and recovery measurements. COW offers strong snapshots and data-integrity capabilities, but can increase write amplification and space retention. Neither journaling nor COW replaces independent backups and application-level consistency.

11. Source References

[1] Microsoft Learn, *File System Functionality Comparison* - FAT32 and NTFS limits and journaling features. https://learn.microsoft.com/en-us/windows/win32/fileio/filesystem-functionality-comparison

[2] Microsoft Learn, *NTFS Overview* - cluster-size-dependent maximum volume and file sizes. https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview

[3] Linux Kernel Documentation, *ext4 General Information* and *ext4 Journal* - data=journal, ordered and writeback modes. https://docs.kernel.org/admin-guide/ext4.html ; https://docs.kernel.org/filesystems/ext4/journal.html

[4] Linux Kernel Documentation, *ext4 Blocks* - block sizes and filesystem/file-size limits. https://kernel.org/doc/html/latest/filesystems/ext4/blocks.html

[5] Linux Kernel Documentation, *XFS* and *XFS Self-Describing Metadata* - journaling, CRC metadata checks. https://www.kernel.org/doc/html/latest/admin-guide/xfs.html ; https://www.kernel.org/doc/html/latest/filesystems/xfs/xfs-self-describing-metadata.html

[6] Btrfs Documentation, *Introduction* and *Administration / Filesystem Limits* - COW, snapshots, checksums and size limits. https://btrfs.readthedocs.io/en/stable/Introduction.html ; https://btrfs.readthedocs.io/en/stable/Administration.html

[7] OpenZFS Documentation, *Copy-on-Write*, *Checksums* and *Snapshots*. https://openzfs.github.io/openzfs-docs/Basic%20Concepts/Copy-on-write.html ; https://openzfs.github.io/openzfs-docs/Basic%20Concepts/Data%20Storage/Checksums.html ; https://openzfs.github.io/openzfs-docs/Basic%20Concepts/Datasets/Snapshots%20and%20Clones.html

[8] Apple Support, *Role of Apple File System* - APFS encryption, COW metadata, snapshots, clones and space sharing. https://support.apple.com/en-in/guide/security/seca6147599e/web

[9] Apple Developer Documentation Archive, *APFS Guide / FAQ* - APFS file-size/addressability information, metadata checksums and no traditional journal. https://developer.apple.com/library/archive/documentation/FileManagement/Conceptual/APFS_Guide/VolumeFormatComparison/VolumeFormatComparison.html ; https://developer.apple.com/library/archive/documentation/FileManagement/Conceptual/APFS_Guide/FAQ/FAQ.html

Source alignment: Assignment 4 - CO4, Task 3 - Modern File Systems, page 36 of the supplied question paper. Subtasks: 3.1 compare modern file systems; 3.2 journaling and consistency; 3.3 choose for each device; 3.4 inspect a real filesystem; plus the COW alternative.